

C++ Programming Assessment – ATM Banking Application

Assessment Requirements

- Create a Project to demonstrate an ATM Banking Application. The program must use OOP concepts, with a Constructor being compulsory, along with the scope resolution operator, loops, and conditional statements wherever required.
- Comments must be included where needed for clarity.
- Display a Welcome Screen with the current date and time, plus a menu (Press 1 to access via PIN, Press 0 for Help). ATM PIN is 12345.
- Choice 1: prompt for the ATM PIN, allowing only ONE attempt.
- Choice 0: display a Help screen.
- Incorrect PIN: exit with a 'Thank You' message screen (no more attempts allowed).
- Correct PIN: display the Main Menu (Deposit Cash, Withdraw Cash, Balance Inquiry, Exit ATM).
- Deposit: show account holder details (name, address, branch, account number), the present balance, prompt for a deposit amount, and display the updated balance. Initial starting balance: Rs. 60000. Present account balance: Rs. 20000.
- Withdrawal: if the balance is insufficient, display an 'Insufficient Available Balance' message.
- Balance Inquiry: display the appropriate balance output.
- After completion, upload the project to GitHub - push all features to the develop branch, then merge into main once complete.

Answer – Full C++ Program (main.cpp)

The program below uses a BankAccount class (OOP), a compulsory constructor, member functions defined outside the class using the scope resolution operator (::), a while loop to keep the main menu running, a switch-case for menu selection, and if-else conditional statements for PIN verification and balance checks. Comments are included throughout for clarity.

```
// =====  
// ATM Banking Application - C++ OOP Assessment Project  
// =====  
#include <iostream>  
#include <ctime>  
using namespace std;  
  
// -----  
// Class: BankAccount  
// Demonstrates OOP - encapsulates account holder details and  
// account balance, along with the operations an ATM performs.  
// -----  
class BankAccount {  
private:  
    string accountHolderName;  
    string accountHolderAddress;  
    string branchLocation;  
    string accountNumber;  
    double initialBalance;    // opening balance when the account was created (Rs. 60000)  
    double currentBalance;    // present available balance (Rs. 20000)  
    static const int correctPin = 12345; // fixed ATM PIN for this assessment
```

```

public:
    // Compulsory constructor - initializes all account details when the object is created
    BankAccount(string name, string address, string branch, string accNum,
                double initBal, double currBal);

    // Member function declarations (defined later using the scope resolution operator)
    bool verifyPin(int enteredPin);
    void displayAccountHeader(const string &title);
    void deposit();
    void withdraw();
    void checkBalance();
};

// -----
// Constructor defined outside the class using the
// Scope Resolution Operator (::)
// -----
BankAccount::BankAccount(string name, string address, string branch, string accNum,
                        double initBal, double currBal) {
    accountHolderName = name;
    accountHolderAddress = address;
    branchLocation = branch;
    accountNumber = accNum;
    initialBalance = initBal;
    currentBalance = currBal;
}

// Verifies the PIN entered by the user against the account's correct PIN
bool BankAccount::verifyPin(int enteredPin) {
    return (enteredPin == correctPin);    // returns true only on an exact match
}

// Displays a reusable header with the account holder's details -
// used by deposit(), withdraw() and checkBalance() so the info isn't repeated
void BankAccount::displayAccountHeader(const string &title) {
    cout << "=====\n";
    cout << "\t\t " << title << "\n";
    cout << "=====\n\n";
    cout << "The Names of the Account Holders are : " << accountHolderName << "\n\n";
    cout << "\t\tThe Account Holders' address is : " << accountHolderAddress << "\n\n";
    cout << "\t\tThe Branch location is : " << branchLocation << "\n\n";
    cout << "\t\tAccount number : " << accountNumber << "\n";
    cout << "-----\n\n";
}

// Handles a cash deposit
void BankAccount::deposit() {
    displayAccountHeader("ATM ACCOUNT DEPOSIT SYSTEM");
    cout << "\t\tPresent available balance : Rs. " << currentBalance << "\n\n";

    double amount;
    cout << "\t\tEnter the Amount to be Deposited Rs. ";
    cin >> amount;

    // Conditional statement: reject an invalid deposit amount
    if (amount <= 0) {
        cout << "\nInvalid deposit amount. Transaction cancelled.\n";
    } else {
        currentBalance += amount;    // update the balance
        cout << "\n\t\tYour new available Balanced Amount is Rs. " << currentBalance << "\n";
        cout << "\n\t\t\t\tThank You!\n";
    }
    cout << "\nPress any key to Return to the Main Menu";
    cin.ignore(); cin.get();
}

// Handles a cash withdrawal
void BankAccount::withdraw() {
    displayAccountHeader("ATM ACCOUNT WITHDRAWAL");
    cout << "\t\tPresent available balance : Rs. " << currentBalance << "\n\n";
}

```

```

double amount;
cout << "\tEnter the Amount to Withdraw Rs. ";
cin >> amount;

// Conditional statement: check sufficient funds before allowing withdrawal
if (amount > currentBalance) {
    cout << "\n\tInsufficient Available Balance in your account.\n";
    cout << "\n\t\t\tSorry !!\n";
} else if (amount <= 0) {
    cout << "\nInvalid withdrawal amount.\n";
} else {
    currentBalance -= amount;
    cout << "\n\tPlease collect your cash.\n";
    cout << "\tYour new available Balance Amount is Rs. " << currentBalance << "\n";
    cout << "\n\t\t\tThank You!\n";
}
cout << "\nPress any key to continue . . . ";
cin.ignore(); cin.get();
}

// Displays the account's balance information
void BankAccount::checkBalance() {
    displayAccountHeader("ATM BALANCE INQUIRY");
    cout << "\tYour Initial Starting Account balance : Rs. " << initialBalance << "\n\n";
    cout << "\tYour Present available balance is : Rs. " << currentBalance << "\n";
    cout << "\nPress any key to continue . . . ";
    cin.ignore(); cin.get();
}

// -----
// Free (non-member) helper functions for the screens that
// don't need direct access to account data
// -----

// Displays the Welcome screen with the current date and time
void displayWelcomeScreen() {
    time_t now = time(0);        // get the current system time
    char* dt = ctime(&now);      // convert it into a human-readable string

    cout << "=====\n";
    cout << "\t\t\tWELCOME TO ATM\n";
    cout << "=====\n\n";
    cout << "\t\t\t-----\n";
    cout << "\tCurrent date : " << dt;
    cout << "\t\t\t-----\n\n";
    cout << "=====\n\n";
    cout << "Press 1 and Then Press Enter to Access Your Account Via Pin Number\n";
    cout << "\t\t\t\tor\n";
    cout << "Press 0 and press Enter to get Help.\n\n";
}

// Displays the Help screen (Choice 0)
void displayHelpScreen() {
    cout << "\n=====\n";
    cout << "\t\t\tATM ACCOUNT STATUS\n";
    cout << "=====\n\n";
    cout << "You must have the correct pin number to access this account.\n";
    cout << "See your bank representative for assistance during bank opening hours.\n\n";
    cout << "\t\t\tThanks for, your choice today!!\n";
    cout << "=====\n";
    cout << "\nPress any key to continue . . . ";
    cin.ignore(); cin.get();
}

// Displays the exit screen shown after an incorrect PIN attempt
void displayThankYouScreen() {
    cout << "\n=====\n";
    cout << "\t\t\t\tTHANK YOU\n";
    cout << "=====\n\n";
    cout << "You had made your attempt which failed!!! No More attempts allowed!! Sorry!!\n";
}

```



```

        cout << "\nInvalid choice. Please try again.\n";
    }
} else {
    // Incorrect PIN -> exit screen with a message, no further attempts
    displayThankYouScreen();
}
}
else {
    cout << "\nInvalid selection. Program terminated.\n";
}

return 0;
}

```

Notes

OOP concepts used: encapsulation (private data members with public member functions), a compulsory constructor that initializes account details on object creation, and the scope resolution operator (`BankAccount::functionName`) to define every member function outside the class body. Control flow uses a while loop to keep the ATM menu running until the user exits, a switch-case to route the menu selection, and if-else conditionals for PIN verification, deposit validation, and the insufficient-balance check on withdrawal. To finish the assessment: compile and run this file (e.g. `main.cpp`) in your C++ environment, test all four menu paths (deposit, withdraw with sufficient and insufficient balance, and balance inquiry) plus both PIN outcomes, then push the code to a 'develop' branch on GitHub and merge it into 'main' once everything is verified, as the assessment's final step requires.